COMPSCI 2XC3 Lab Report 2

Prepared by

Group 64

Luca Mawyin

Anderson Ray

Theo Pham

COMPSCI 2ME3

McMaster University

February 22, 2026

Contents

Part 1 3
Experiment 1 3
Experiment 2 3

Part 2 5
Experiment 3 5
Experiment 4 5
Experiment 5 6
Worst Case of Approximation 7
Independent Set Problem 7

List of Figures

1 Probabilties of Cycles Existing in Graphs With 200 Nodes 3
2 Probabilties of Cycles Existing in Graphs With 100 Nodes 4
3 Vertex Cover Size in Graphs of Increasing Edges 5
4 Relative Vertex Cover Size in Graphs of Increasing Edges 6
5 Vertex Cover Size in Graphs of Increasing Nodes 6

Part 1

Experiment 1

In our experiment finding probabilties of cycles existing in graph, we used the following parameters:

- 200 Nodes
- ranging from 0 to 100 edges
- running 10000 tests per edge amount

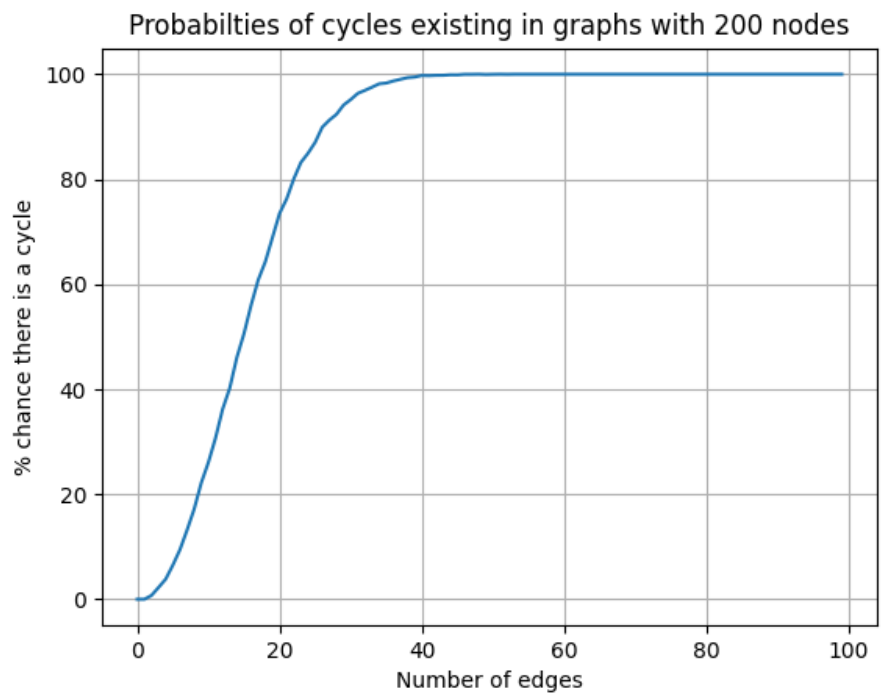

Figure 1: Probabilties of Cycles Existing in Graphs With 200 Nodes

In Figure 1 at around 40 edges, the graph tops out meaning that it's very likely that a graph with 200 nodes and over 40 random edges will have a cycle. This is because each time a random edge is added, the chances of the edge creating a cycle increases, resulting in the S shaped curve in Figure 1.

Experiment 2

In our experiment finding probabilties of graphs being connected, we used the following parameters:

- 200 Nodes
- ranging from 0 to 1500
- doing 1000 tests every 10 edge amounts (0 edges, 10 edges, 20 edges, ..., 1000 edges)

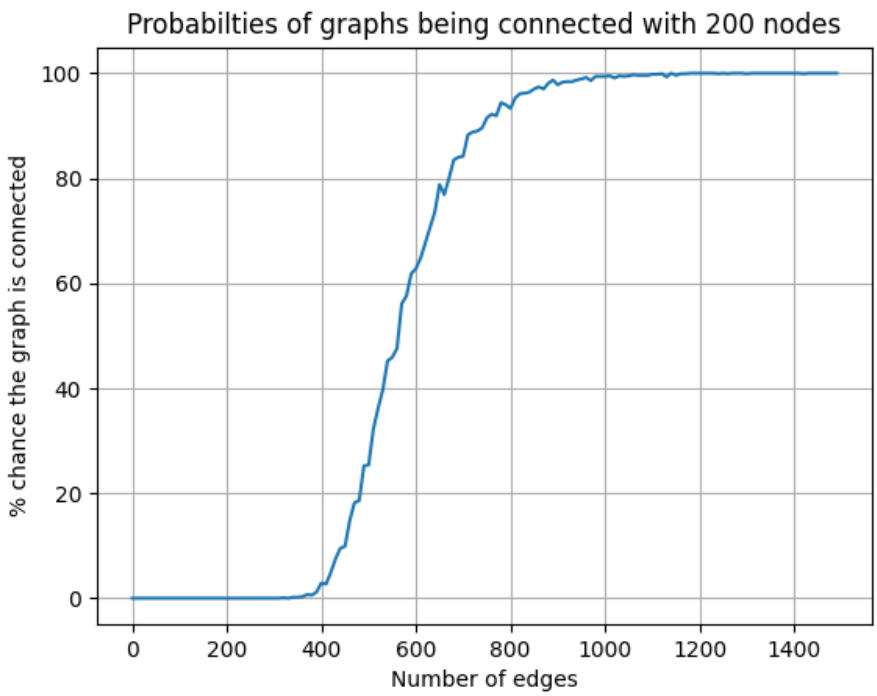

Figure 2: Probabilties of Cycles Existing in Graphs With 100 Nodes

In Figure 2 at around 1000 edges, the graph tops out meaning that it's very likely that a graph with 200 nodes and over 1000 random edges will be connected. The graph with the max amount of edges possible without making it connected would look like a graph where every node is connected to every other node except for one specific node. To get this max number of edges, you set N = number of nodes and plug it into $\frac{(N-1)(N-2)}{2}$, which gives us 19701 for our experiment. Our experiment tops out much before this max though as the chances of fully connecting the entire graph with a single unique edge increase as the graph has more and more existing edges. This makes it very unlikley for more denser graphs($\geq$ 1000 edges) to be unconnected.

In Figure 2 the graph bottoms out until about 300 edges. This is because a graph cannot be connected if: number of edges $<$ number of nodes $-$ 1 and even if you have more edges then that if they are picked at random it's very unlikely for it to be connected.

Part 2

Experiment 3

In order to test the approximation algorithms, we used the following parameters:

- Generate 1000 random graphs with 8 nodes and m edges, with incrementing edges from 1 to 30
- Find the MVC for each graph and compare to the execution of each of the approx functions

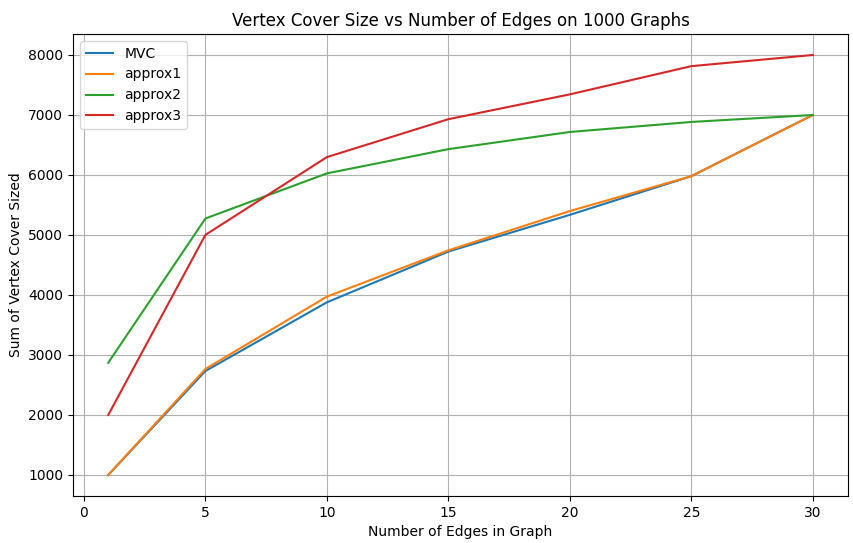

Figure 3: Vertex Cover Size in Graphs of Increasing Edges

In Figure 3 the approx1 function seems to approximate the Vertex Cover Size (VCS) to be very similar to the Minimum Vertex Cover (MVC). This is likely due to the fact that the algorithm functions on selecting the node with the highest number of edges, which often results in a near-minimal set of edges. The other two algorithms result in a VCS substantially larger than the MVC, likely due to the randomness factor, which makes more optimal vertex selections less likely. However, as the number of edges in the graph increases, this disparity between the MVC and approx2 and approx3 decreases, likely because the concentration of edges is so high that it is difficult to make a poor decision on vertex selection, even with a randomness factor.

Experiment 4

This experiment better highlights the performance difference between each of the approximation algorithms. Instead of comparing all the Vertex Cover Sizes, each algorithm was compared to the MVC. Therefore, the relative performance of each algorithm can be better analyzed. All of the tests were run with the same parameters as Experiment 3

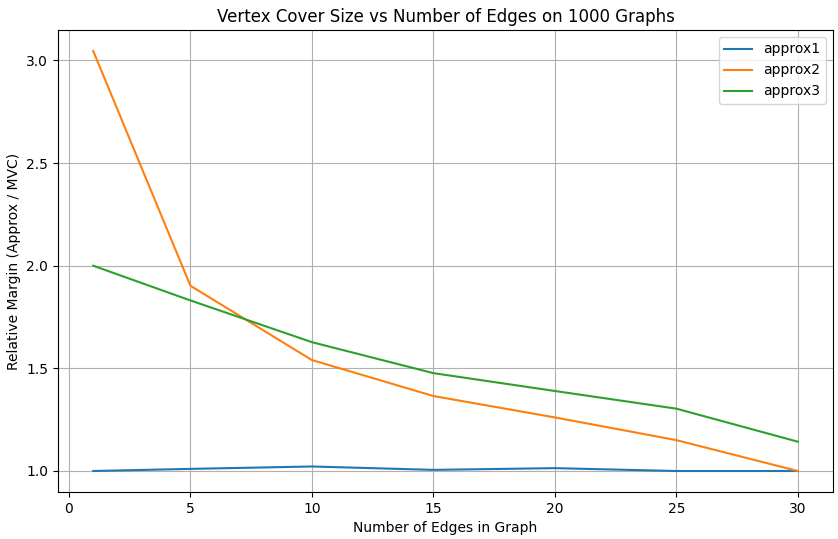

Figure 4: Relative Vertex Cover Size in Graphs of Increasing Edges

As shown in Figure 4, the relative VCS of each algorithm decreases as the number of edges in the graph increases. This is due to the fact that the higher concentration of edges relative to the number of nodes in the graph make it more difficult to select a "bad" edge.

Experiment 5

We can also test the approximation algorithms with varying number of nodes instead of edges. This experiment compares the VCS of all the algorithms (with the MVC) with varying number of nodes. In order to maintain a reasonable execution time, the number of edges allowed in a graph was half of the maximum number of total possible edges in a graph. The testing was run on 1000 different graphs, with each algorithm using the same graph as each other.

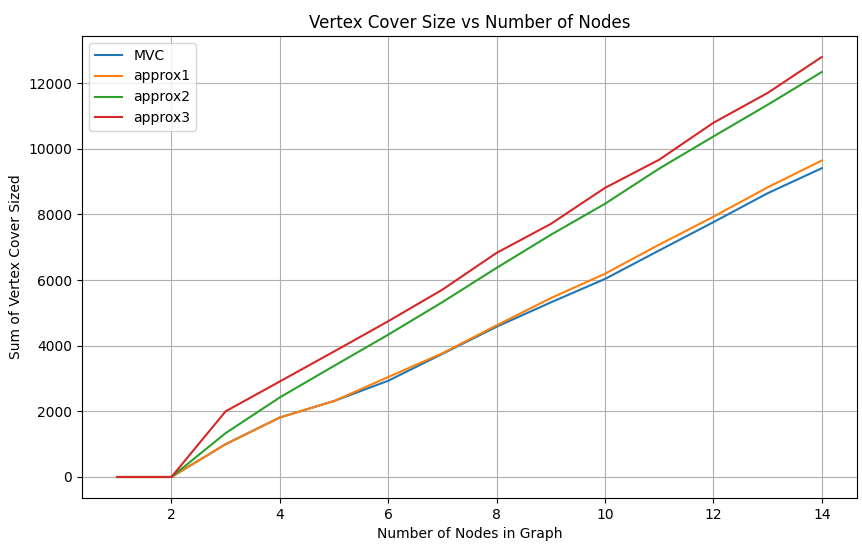

Figure 5: Vertex Cover Size in Graphs of Increasing Nodes

Analyzing Figure 5, the graph shows a linear trend for each algorithm. This is because as the node size is increasing, the number of edges increased proportionally. This means that the Vertex Cover Size will maintain the same relative size to the results of the rest of the algorithms.

Worst Case of Approximation

In order to test the worst case for approx1() on a small graph of five nodes, you would have to calculate all possible graphs with 5 nodes. The maximum number of edges in a graph is expressed as:

$$\begin{aligned}\text{max number of edges} &= \frac{n(n-1)}{2} \\ &= \frac{5(5-1)}{2} \\ &= \frac{20}{2} \\ &= 10\end{aligned}$$

Therefore, a graph with five nodes can have at most ten edges. Furthermore, within a graph an edge can either exist or not exist. This means that we can express the total number of graphs as 2^{10}, as for every max number of edges in the, there exist two options for an edge's existance. This means that there are 1024 total possible graphs of size ten. In order to test for this, you can enumerate each subset of edges in a graph.

Independent Set Problem

When understanding the basis of the Independent Set Problem, the relation between the Minimum Vertex Cover (MVC) and Maximum Independent Set (MIS) is extremely intuitive. The MIS is the largest set of nodes in a graph with edges that do not connect. This is effectively the opposite of the MVC, as the MVC is the smallest set of nodes in a graph that connects every to every edge. Subsequently, this would mean that set of nodes in MVC and the set of nodes in MIS converge such that $\text{MVC} \cup \text{MIS} = G$, where G is the set of all nodes in the graph. Furthermore, in order to implement an MIS function, we can simply apply inverted logic to the already-existing MVC function.

```
def is_independent_set(G, C):
    for start in G.adj:
        for end in G.adj[start]:
            if start in C and end in C:
                return False
    return True

def MIS(G):
    nodes = [i for i in range(G.number_of_nodes())]
    subsets = power_set(nodes)
    maxSet = []
    for subset in subsets:
        if is_independent_set(G, subset):
            if len(subset) > len(maxSet):
                maxSet = subset
    return maxSet
```